

• h5i • PITCH

Is the Web App Your AI Built Really Secure?

AI agents now write and ship web apps. h5i is the workspace where developers and their AI check that what shipped is correct and secure.

■ SECURITY HAS TWO SIDES

Find vulnerabilities. Prove correctness.

Two sides of one coin.

01 · Red-teaming

Find vulnerabilities

Attack the running app the way a hacker would, and catch what nobody anticipated.

Expert hackers are expensive. Red-teaming AI can go rogue.

ONE



COIN

02 · Formal verification

Prove correctness

State the rules the logic must keep, and prove the code satisfies them.

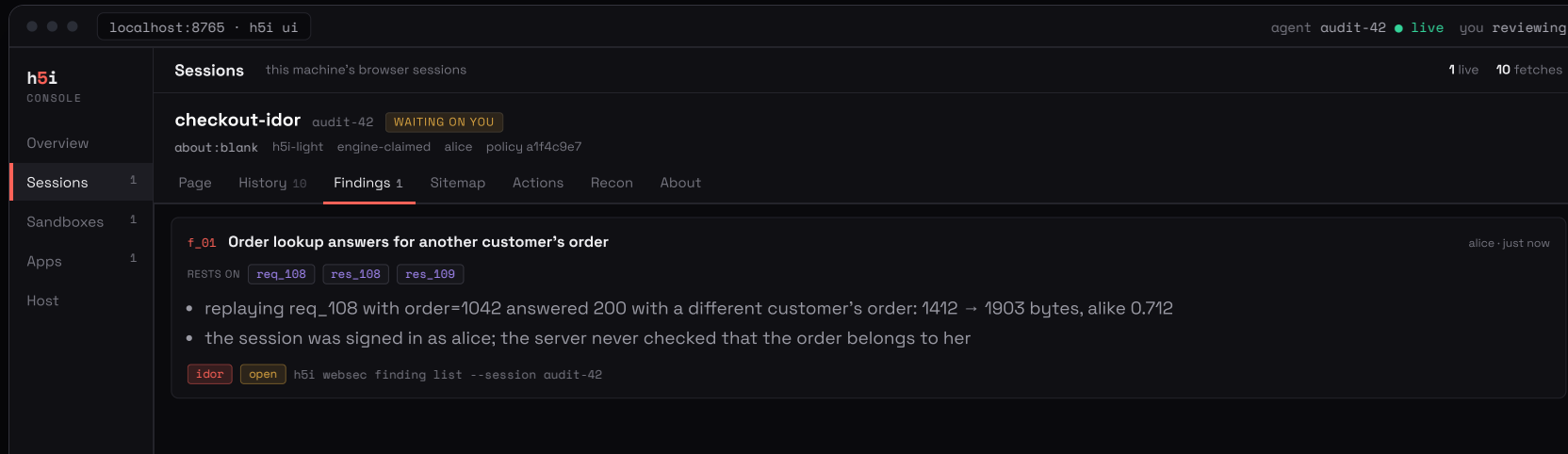
Writing code is easy. Proving it correct is hard.

You need both, with every change the AI ships.

■ WHAT H5I IS

One workspace for both, for developers and their AI.

The AI runs one CLI. You review one console: findings with evidence, theorems with receipts.



■ FIND VULNERABILITIES / ANY STACK

Your agent red-teams the running app.

It drives the app like a user and replays real requests like a hacker, inside limits you set.

h5i
CONSOLE

Overview

Sessions 1

Sandboxes 1

Apps 1

Host

localhost:8765 · h5i ui

agent audit-42 ● live you reviewing

Sessions this machine's browser sessions 1 live 10 fetches 1 refused

checkout-idor audit-42 LIVE 1 REFUSED

about:blank h5i-light engine-claimed alice policy a1f4c9e7

Page History 10 Findings 0 Sitemap Actions Recon About

#	verb	method	host	path	status	time	initiator
107	open	GET	shop.example	/account	200	412ms	navigation
108	click	GET	shop.example	/api/order/1041	200	31ms	subresource
109	replay	GET	shop.example	/api/order/1042	200	29ms	replay

Lean 4 proves your logic correct.

Write the logic on h5i-app. Aeneas extracts it into Lean 4, and the Apps view shows what is proven under which assumptions. Red-teaming covers the rest.

localhost:8765 · h5i ui

agent audit-42 ● live you reviewing

h5i
CONSOLE

Overview

Sessions 1

Sandboxes 1

Apps 1

Host

Apps Lean proofs in this repository: what is proven, what is assumed, and what is unconfirmed 1 proof project 1 with a receipt 0 reporting a failure

board examples/app/tutorials/board RECEIPT - JUST NOW
kernel/src/lib.rs → BoardKernel.lean Charon + Aeneas h5i-app 0.1

Spec **Theorems 3** Trust boundary Evidence Counterexamples

Each row is a statement about the extracted kernel. Before it says anything about the running app, the trust boundary must hold.

theorem	statement	status	mutants
authorized Theorems.lean:79	Every write a successful command makes is allowed by the policy.	proven	4 of 4 caught
inv_preserved Theorems.lean:160	Invariants hold in every state the board can reach.	proven	5 of 5 caught

■ WHY H5I

Other tools give you one side. h5i gives you the coin.

Browser + proxy tools

Built for people. The agent juggles browser, proxy, and CA setup. No proofs.

Sandboxes

Limit what the agent can reach. No testing, no evidence, no proofs.

Proof assistants

Prove a model of the logic. Nothing connects it to the running app.

h5i

Red-team the running app and prove the logic, in one CLI the AI runs and one console you review.

■ TRACTION & USERS

Built in the open.

Built for teams that ship with AI.

670+

GitHub stars

60+

forks

20+

contributors

2.5k+

release downloads

GitHub · checked October 2, 2026 · [Repository and releases](#)

Product teams

shipping with AI, no security specialist

Security engineers

testing between assessments, with evidence

Rust teams

with rules that must never break

■ NEXT

Find vulnerabilities. Prove correctness. For every team that ships with AI.

Product

findings beside the proofs they touch, in one console

Adoption

10 design partners, then the first paid pilots

Open source

local-first, Apache 2.0, built with the community

Try it with your own AI agent: `curl -fsSL https://h5i.dev/install.sh | sh -s -- --websec --test`

h5i.dev

github.com/h5i-dev/h5i